

Fiche de procédure

Angular

Cours

OpenClassrooms

En bref, [Angular](#) est un **framework JavaScript** qui vous permet de développer des applications "efficaces et sophistiquées", comme dit la documentation. Il permet notamment de créer ce qu'on appelle des **Single Page Applications** (ou **SPA**) : des applications entières qui tournent dans une seule page HTML grâce au JavaScript.



Un **framework** logiciel est un ensemble d'outils et de composants à partir desquels on peut développer des applications.

Le développement Angular passe par trois langages principaux :

- le **HTML** pour structurer – toutes vos connaissances avec ce langage vous seront utiles, et Angular viendra vous ajouter quelques nouveautés ;
 - le **SCSS** pour les styles – le SCSS est une surcouche du CSS qui y apporte des fonctionnalités supplémentaires, mais qui permet également d'écrire du CSS pur si on le souhaite ;
 - le **TypeScript** pour tout ce qui est dynamique, comportement et données – un peu comme le JavaScript sur un site sans framework.
-
- Angular est un framework **complet** – on peut créer des applications web complètes sans avoir besoin de librairies tierces supplémentaires. C'est notamment ce qui différencie un *framework* d'une *library*.
 - Les **best practices** (les bonnes pratiques) Angular sont extrêmement précises – les structures des applications Angular ont tendance à se ressembler fortement, donc il est facile de passer d'un projet à un autre, et de s'assurer que son propre projet suit les meilleures pratiques.
 - Le framework a été conçu pour fonctionner avec le **TypeScript** – il est tout à fait possible d'utiliser le TypeScript pour React, Vue ou Svelte, mais Angular a été conçu pour ce langage, donc son intégration est forcément plus profonde.

En résumé

- Pour installer le CLI d'Angular, il vous faut une version LTS de Node et npm.
- On installe le CLI avec la commande `npm i -g @angular/cli`
- On peut vérifier la version installée du CLI avec `ng v`



ng new → nouvelle application.

-style css → utilisé des styles scss...

-routing true → implémentation automatique tout ce qui concerne le routing (routes)

-minimal true → ne pas inclure les tests unitaires frameworks

-ssr false → server side renderReview false

-zoneless false → app ne va plus implémenter [zone.js](#) tout le temps donc faire attention aux signaux.

-strict false → Permettre de mettre du null dans un string (string?) au lieu d'écrire let test number | null | undefined | unknown | any.

-inline-template false

-inline-style false

-ai-config none → désactiver génération des prompts ia

ng serve → exécute le serveur de dev.

ng generate component "nom-component" = → Créer un component.

Implémenter ngZorro et TailWind :

ngZorro : ng add ng-zorro-antd

Voc :

MVC → Model View Controller

Formulaire ngzorro :

- Une application Angular peut être vue comme une arborescence de components.
- Un component lie un template HTML et des styles SCSS à du comportement TS.
- Pour créer un component, on utilise la commande

```
ng generate component nom-du-component
```
- Le sélecteur d'un component correspond à la balise HTML personnalisée qu'on utilisera pour l'insérer dans l'application.

- On déclare les propriétés d'un component en haut de sa classe, et on les initialise dans la méthode `ngOnInit()`
- La méthode `ngOnInit()` est appelée une fois par instance de component au moment de la création de cette instance.
- La string interpolation avec les doubles accolades `{{ }}` permet d'insérer la valeur d'une propriété TypeScript dans le template.
- L'attribute binding permet de lier la valeur d'une propriété TypeScript à un attribut HTML, en mettant l'attribut entre crochets `[]` et en passant le nom de la propriété.
- Pour lier une méthode à un événement d'un élément du template, mettez l'événement entre **parenthèses** `()` et passez la méthode en argument ; ex. : `(click)="onClickButton()"`

Angular vous permet d'ajouter des **propriétés personnalisées** à vos components de manière à ce que vous puissiez les injecter **depuis le component parent**. Ce que ça vous permettrait de faire avec `FaceSnapComponent`, ce serait de faire en sorte que `AppComponent` centralise les données de plusieurs FaceSnaps, génère une instance du component pour chaque FaceSnap, et injecte ce FaceSnap à cette instance.

Mais avant de voir comment créer ce type de propriété, profitons de ce moment pour créer votre premier **type personnalisé** sous forme de classe. On n'a pas envie de devoir injecter un titre, une description, une URL, etc., manuellement à chaque component ! Ce qui serait plus utile, ce serait un **modèle de données** `FaceSnap`, qui comporterait toutes ces propriétés, et que vous pourriez injecter directement dans `FaceSnapComponent`, et même utiliser partout dans votre application.

Heureusement, TypeScript vous permet de faire exactement ça !

Dans votre dossier `app`, créez un sous-dossier `models`, et dedans, créez un fichier `face-snap.ts` dans lequel vous allez créer une classe FaceSnap. Cette classe comportera toutes les propriétés nécessaires d'un FaceSnap. Ça pourrait être intéressant également d'implémenter un **constructor** qui faciliterait la création d'objets FaceSnap.

Je vous propose d'essayer d'implémenter cette classe **vous-même**. Vous trouverez ma proposition ci-dessous :

- N'hésitez pas à créer vos propres types, sous forme de classe par exemple, pour faciliter la manipulation de données dans votre application.
- Une propriété personnalisée est rendue injectable depuis l'extérieur grâce au décorateur `@Input()`
- Une propriété en `@Input()` est utilisable comme n'importe quelle autre propriété : on peut en afficher les éléments, les modifier...
- On lie ensuite une valeur à cette propriété depuis le composant parent avec l'attribut `attribut`, c'est-à-dire le nom de la propriété entre crochets `[]` en passant la valeur entre les guillemets ; ex. : `[faceSnap]="mySnap"`
- Vous pouvez ajouter des propriétés optionnelles à une classe en y ajoutant un point d'interrogation `?`
- Ajouter un control flow block `@if` dit à Angular d'ajouter ou non son contenu selon la condition que vous lui passez, par exemple `@if (size === 'large')`
- le control flow block `@for` fonctionne un peu comme une boucle `for...of`, c'est-à-dire que, dans ce cas de figure, elle va itérer sur le tableau `faceSnaps` et vous exposer l'élément correspondant à chaque itération sous l'alias `faceSnap`
- vous associez ensuite l'élément du tableau à la propriété `faceSnap` du composant
- le paramètre `track` permet à Angular d'identifier les différents éléments du tableau grâce à une propriété unique — puisque notre modèle FaceSnap n'a pas encore de propriété `id`, on utilisera le `title` comme propriété unique (pour notre exemple simple où l'on sait que c'est le cas !)
- Le control flow block `@for` vous permet d'insérer un élément dans le DOM pour chaque élément dans un tableau.
- La syntaxe `element of array` vous donne accès à l'élément de chaque itération, vous permettant de l'injecter dans un composant, par exemple.
- Le paramètre `track` demande une propriété qui sera unique pour chaque élément du tableau (qui sera souvent mais pas toujours son `id`, s'il en a un)

- La directive `[ngStyle]` permet d'appliquer des styles qui dépendent de valeurs dynamiques, comme un texte qui, plus il y a de snaps, devient de plus en plus vert.
- La directive `[ngStyle]` prend comme argument un objet où les clés correspondent aux styles CSS à modifier, et les valeurs aux valeurs que doivent prendre ces styles.
- La directive `[ngClass]` permet d'ajouter dynamiquement des classes à des éléments HTML.
- `[ngClass]` prend un objet en argument où les clés sont des noms de classe CSS, et les valeurs sont les conditions selon lesquelles les classes doivent être appliquées.

Qu'avez-vous appris dans cette partie du cours ?

- Vous avez découvert les directives : des classes qui permettent d'ajouter des comportements à des éléments HTML, ou même à des composants.
- Vous avez implémenté des directives structurales pour modifier la structure du document :
 - `*ngIf` pour ajouter un élément ou non au DOM selon une condition donnée ;
 - `*ngFor` pour ajouter autant d'éléments au DOM qu'il y a d'éléments dans un tableau.
- Vous avez utilisé des directives par attribut pour gérer dynamiquement les styles :
 - `[ngStyle]` pour paramétrer des styles selon des valeurs venant du TypeScript ;
 - `[ngClass]` pour ajouter et retirer des classes CSS selon une condition donnée.

Modifiez la casse

Il existe trois pipes fournis par Angular pour modifier la casse :

- **LowerCasePipe** : on affiche le texte en **minuscules**
 - **UpperCasePipe** : on affiche le texte en **majuscules**
 - **TitleCasePipe** : on affiche le texte avec une majuscule au début de chaque mot, avec le reste du mot en minuscules
- Un pipe permet de formater l'affichage d'une donnée sans modifier la donnée sous-jacente.
 - Angular fournit trois pipes pour modifier la casse affichée d'une chaîne de caractères : UpperCasePipe, LowerCasePipe, et TitleCasePipe pour les majuscules, les minuscules et le mélange type "titre".

Dans le cadre de ce cours, je voudrais partager avec vous une dernière catégorie de pipes : ceux qui concernent le formatage des **nombres** selon les règles de locale de l'application. Il y en a trois :

- **DecimalPipe** – facilite l'affichage de nombres avec des **chiffres après la virgule** (qui met une virgule plutôt qu'un point, par exemple).
 - **PercentPipe** – formate les chiffres en **pourcentage**.
 - **CurrencyPipe** – permet d'afficher des nombres sous forme de **monnaie** très facilement.
-
- DecimalPipe (mot-clé `number`) vous permet d'afficher un nombre selon les règles de la locale de l'application.
 - PercentPipe affiche un nombre entre 0 et 1 sous forme de pourcentage, arrondi par défaut à l'entier le plus proche.
 - DecimalPipe et PercentPipe acceptent un argument de configuration sous la forme `'chiffresMinAvantVirgule.chiffresMinAprèsVirgule-chiffresMaxAprèsVirgule'` ex. : `'1.0-1'`
 - CurrencyPipe facilite l'affichage des montants d'argent, et accepte plusieurs arguments de configuration séparés par des deux-points `:`

Qu'avez-vous appris dans cette partie du cours ?

- Vous avez découvert les pipes – des outils pour gérer le format d'affichage d'une donnée sans modifier la donnée elle-même.
 - Vous avez changé la casse de vos chaînes de caractères avec UpperCasePipe, LowerCasePipe et TitleCasePipe.
 - Vous avez modifié l'affichage de la date de création des FaceSnaps avec DatePipe, et vous avez vu les différentes manières de personnaliser l'affichage des dates en passant un argument de configuration à DatePipe.
 - Vous avez manipulé le format d'affichage des nombres avec DecimalPipe, PercentPipe et CurrencyPipe.
-
- Les services permettent de centraliser les données et la logique pour les différents domaines de votre application.
 - Créer un service est aussi simple qu'ajouter le décorateur `@Injectable()` à une classe.
 - Pour injecter un service dans un component, ajoutez un argument au constructor du component qui a le type du service, par exemple `private userService: UserService`



Outils utilisés : OpenApi generator.

IDE

FRONT → services => vont call les routes

→ DTO => request/response

Dans le projet Angular :

npm install @openapitools/openapi-generator-cli

npx openapi-generator-cli

récup le fichier openapi-generator-yaml sur le git

Ouvrir le package.json

-rubrique scripts

Rajouter la ligne "openapi" : "rm -rf src/app/services/api && openapi-generator-cli generate -i <http://localhost:4200/swagger/v1/swagger.json> -g typescript-angular -o src/app/service/api

npm run openapi

ngAfterViewInit

ngOnDestroy

CORS : Cross

Origin

Ressource

Sharing

```
builder.Services.AddCors(options =>
```

```
{
```

```
    options.AddDefaultPolicy(policyBuilder =>
```

```
{
```

```
        policyBuilder.
```

```
            .WithOrigins("http://localhost:4200")
```

```
            .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE")
```

```
            .AllowAnyHeader();
```

```
    });
```

```
});
```

```
tout en bas : app.UseCors();
```